Семинар 6. Распараллеливание циклов и таски в OpenMP

1 Директива for

Вернемся к рассмотрению программы для вычисления значения определенного интеграла с предыдущего семинара. Ее можно упростить, использовав директиву `for`. Получившаяся программа показана в листинге 1.

Листинг 1: Вычисление определенного интеграла

```
1  int main(int argc, char** argv){
2          int thread_count = strtol(argv[1], NULL, 10);
3          double a = 0.0;
4          double b = 3.0;
5          int n = 1000;
6          double h = (b-a)/n;
7          double approx = 0.5*(f(a) + f(b));
8  #pragma omp parallel for num_threads(thread_count)\
9                  reduction(+: approx)
10         for(int i = 1; i < n; ++i)
11                 approx += f(a+i*h);
12         printf("%lf\n", h*approx);
13 }
```

Комментарии.

1. Представленная программа отличается от аналогичной программы без какого-либо распараллеливания лишь наличием строк 2 и 8.

2. В строке 8 дается команда на распараллеливание цикла `for`. Распараллеливание происходит следующим образом. Все итерации цикла

делятся на `thread_count` блоков по ≈`n/thread_count` итераций в каждом, по одному блоку на каждый поток. Затем потоки одновременно выполняют итерации. После завершения цикла каждый поток заходит в неявный барьер. Таким образом, следующая инструкция после цикла будет выполняться только главным потоком и после того, как все остальные потоки завершили свою работу.

3. За директивой `#pragma omp parallel for` может следовать лишь один оператор — цикл `for` в стандартной форме. Стандартная форма означает, что в круглых скобках в первой секции стоит инициализация целочисленной переменной или указателя, вторая секция представляет собой сравнение счетчика цикла с неким значением, а третья секция является операцией увеличения или уменьшения значения счетчика.

4. Цикл не может содержать в себе операторов `break` и `return`, однако вызов `exit` допустим.

5. Счетчик цикла всегда является приватной переменной, даже если объявлен до начала цикла.

2 Директива `schedule`

У программиста есть возможность влиять на то, как именно итерации цикла будут распределяться между потоками. Для этого в директиве `for` нужно указать команду `schedule(type, chanksize)`, где `type` — это тип планирования цикла, а `chanksize` — натуральное число.

Допускаются следующие типы планирования цикла:

1. `static`. Итерации распределяются между потоками при входе в цикл. Каждый поток получает `chanksize` идущих подряд итераций цикла.
   Так, если `chanksize=1`, количество итераций равно 12, а количество потоков трем, то первый поток получит итерации 0,3,6, 9, второй поток получит итерации 1, 4, 7, 10, а третий 2, 5, 8, 11. Такое разбиение называется циклическим.
   Если `chanksize=2`, то первый поток получит итерации 0, 1, 6, 7, второй 2, 3, 8, 9 третий 4, 5, 10, 11. Такое разбиение называется

блочно-циклическим.
Если `chanksize` не указать, то он принимается по умолчанию равным количеству итераций, деленному на количество потоков. Такое разбиение называется блочным.

2. `dynamic`. Каждый поток получает `chanksize` идущих подряд итераций. Если `chanksize` не указать, то он принимается равным 1. После того, как поток выполнил очередной блок итераций, он получает следующий блок, и так далее, пока все итерации не будут выполнены. Важно понимать, что распределение итераций осуществляется по мере того, как потоки справляются с работой. Таким образом, если одни итерации выполняются существенно быстрее других, то тип планирования цикла `dynamic` позволяет обеспечить рациональную балансировку нагрузки, дабы избежать простаивания потоков.

3. `guided`. Аналогично `dynamic`, однако размер блока итераций уменьшается в два раза с каждой выдачей итераций потоку. Значение `chanksize` при этом задает минимальный размер блока, меньше которого может быть только блок в последней выдаче итераций. Если `chanksize` не указан, считается равным 1.
Пусть имеется два потока и 10 000 итераций. Тогда в первой выдаче будет 5000 итераций, которые достанутся нулевому потоку. Во второй выдаче будет уже 2500 итераций, который пойдут первому потоку. Далее освободившийся поток получит 1250 итераций, и т.д.

4. `runtime`. В этом случае `chanksize` не указывается. Может быть любым из вышеперечисленных методов планирования цикла в зависимости от значения переменной окружения `OMP_SCHEDULE`. См. листинги 3, 2.

Листинг 2: Демонстрация использования `schedule(runtime)`

```
int main() {
    int n = 10;
    #pragma omp parallel for num_threads(NTHREADS) \
            reduction(+:sum) schedule(runtime)
    for (int i = 0; i < n; i++) {
            printf("Thread %d: i=%d\n", omp_get_thread_num(), i);
```

```
        }
}
```

Листинг 3: Скрипт для запуска программы с листинга 2

```
#!/bin/bash

schedules=("static dynamic guided")
chunk_sizes=("4 2 1")

for schedule in $schedules
do
        for chunk in $chunk_sizes
        do
                echo $schedule $chunk
                export OMP_SCHEDULE="$schedule, $chunk"
                ./main
        done
done
```

Резюме. Если итерации цикла требуют для выполнения примерно одинаковое количество времени, то лучше не использовать команду `schedule`. Если время выполнения различных итераций цикла может отличаться, следует использовать статическую планировку цикла с блочно-циклическим или циклическим разбиением. Можно попытаться увеличить скорость работы с помощью динамической планировки, но это приведет к дополнительным накладным расходам. Если доподлинно известно, что время выполнения растет с ростом номера итерации, то нужно использовать `guided` планировку. Планировка `runtime` может помочь выбрать нужную планировку в процессе отладки или межпрограммного взаимодействия.

3 Зависимость между итерациями цикла

Запись цикла в стандартной форме еще не гарантирует успешное его распараллеливание. Необходимо следить за тем, чтобы различные итерации цикла не зависели друг от друга. Данную проверку компилятор не выполняет. В случае, если итерации цикла зависят друг от друга, результат работы программы будет недетерминированным.

Задача 1. Написать программу для параллельного нахождения чисел Фибоначчи.

В листинге 4 показана неудачная попытка реализации такой программы. В самом деле, каждая итерация цикла для определения очередного числа Фибоначчи зависит от результатов выполнения двух предыдущих итераций. Если, например, итерация 5 выполняется вторым потоком, а итерации 3 и 4 первым, то, в случае, если первый поток будет выполнять свои команды чуть позже второго, то второй поток не сможет корректно определить пятое число Фибоначчи.

Листинг 4: Неудачная попытка посчитать числа Фибоначчи

```
int main(int argc, char** argv){
        int thread_count = strtol(argv[1], NULL, 10);
        int n = 10;
        int fibs[n];
        fibs[0] = fibs[1] = 1;
#pragma omp parallel for num_threads(thread_count)
        for(int i = 2; i < n; ++i)
                fibs[i] = fibs[i-1] + fibs[i-2];
        for(int i = 0; i < n; ++i)
                printf("%3d", fibs[i]);
        printf("\n");
}
```

Задача 2. Написать программу для параллельного определения числа π как суммы ряда $4\left(1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\ldots\right)$.

Программа, решающая эту задачу, приведена в листинге 5. Программа написана таким образом, чтобы итерации цикла были бы независимыми друг от друга. Также следует обратить внимание на то, что переменная `factor` объявлена как приватная для потока. Если этого не сделать, то возникнет состояние гонки потоков.

Листинг 5: Вычисление числа π

```
1 int main(int argc, char** argv){
2
3         int thread_count = strtol(argv[1], NULL, 10);
4         double factor;
5         double sum = 0.0;
6         int n = 10000;
```

```
7           #pragma omp parallel for num_threads(thread_count)\
8                                   reduction(+: sum)\
9                                   default(none) private(factor) shared(n)
10          for(int i = 0; i < n; ++i){
11                  factor = i % 2 == 0 ? 1.0 : -1.0;
12                  sum += factor/(2*i+1);
13          }
14          printf("%lf\n", 4*sum);
15
16  }
```

Задача 3. Написать программу, осуществляющую параллельную пузырьковую сортировку массива.

Программа для сортировки массива пузырьком состоит из двух вложенных циклов `for`. Итерации обоих этих циклов не являются независимыми. Однако мы можем сделать независимыми итерации внутреннего цикла, внеся модификацию в алгоритм. Для этого на четных итерациях внешнего цикла (фазах) будем сравнивать и менять местами элементы массива, стоящие на четных местах и справа от них. На нечетных итерациях будем сравнивать и менять местами элементы массива, стоящие на нечетных местах и справа от них.

Программа показана в листинге 6.

Обратите внимание на то, что в строке 1 создаются потоки, а затем, в строках 4 и 13, они на каждой итерации внешнего цикла переиспользуются. Это сделано для того, чтобы избежать накладных расходов, связанных с созданием и завершением потоков.

Листинг 6: Четно-нечетная пузырьковая сортировка

```
1  #pragma omp parallel num_threads(thread_count)
2  for(int phase = 0; phase < n; ++phase)
3           if(phase % 2 == 0)
4  #pragma omp for
5                   for(int i = 1; i < n; i += 2){
6                           if(a[i-1] > a[i]){
7                                   int tmp = a[i-1];
8                                   a[i-1] = a[i];
9                                   a[i] = tmp;
10                          }
11          }
```

```
12          else
13 #pragma omp for
14          for(int i = 1; i < n-1; i += 2){
15                  if(a[i] > a[i+1]){
16                          int tmp = a[i];
17                          a[i] = a[i+1];
18                          a[i+1] = tmp;
19                  }
20          }
```

4 Таски в OpenMP

Директива `task` позволяет добиться того, чтобы фрагмент кода, составляющий таску, выполнялся бы первым освободившимся потоком. Данную директиву нужно писать внутри директивы `parallel`, то есть когда потоки уже созданы. Также обычно таски создаются внутри директивы `single`, которая заставляет код выполняться только одним каким-то потоком. Без нее все потоки выполнили бы таску каждый по одному разу.

Директива taskwait реализует барьер, в котором мы ждем, когда все таски, созданные в текущей таске, завершат свою работу.

В качестве примера применения тасков рассмотрим параллельное вычисление чисел Фибоначчи с помощью рекурсии, см. листинг 7.

Обратите внимание на то, что в строках 15 и 17 мы обозначаем переменные `i,j` как относящиеся к общей памяти. Дело в том, что, хоть они и объявлены до таски, по умолчанию считаются приватными внутри таски. Поэтому, не напиши мы слово `shared`, получили бы нули или мусор в ответе.

В тех же строках 15 и 17 мы используем конструкцию `if(n>20)`, что означает: создавать новую задачу только тогда, когда планируется большой объем работы, а именно, `n` больше 20. Иначе нет смысла тратить ресурсы на добавление задачи в планировщик и выделение для нее ресурсов. Число 20 подобрано эмпирически, так, чтобы получить хоть какое-то ускорение.

Листинг 7: Параллельное вычисление чисел Фибоначчи

```
1 #include <omp.h>
```

```
 2 #include <stdio.h>
 3 #include <stdlib.h>
 4
 5 int fibs[100];
 6
 7 int fib(int n){
 8
 9         int i = 0;
10         int j = 0;
11         if(n <= 1){
12                 fibs[n] = n;
13                 return n;
14         }
15 #pragma omp task shared(i) if(n > 20)
16         i = fib(n-1);
17 #pragma omp task shared(j) if(n > 20)
18         j = fib(n-2);
19 #pragma omp taskwait
20         fibs[n] = i + j;
21         return fibs[n];
22
23 }
24
25 int main(int argc, char** argv){
26
27         int thread_count = strtol(argv[1], NULL, 10);
28         int n = strtol(argv[2], NULL, 10);
29
30 #pragma omp parallel num_threads(thread_count)
31 #pragma omp single
32         fib(n);
33         printf("%lf\n", end-start);
34         for(int i = 0; i < n; ++i)
35                 printf("%d\n", fibs[i]);
36 }
```